1. Basic experiment
    a. My Fitness Function
        i. Equation: the maximum value – length between my creature and the mountain peak
            1. The maximum value is 20
            2. The minimum value is 0
            3. If the length between the creature and the mountain peak is larger than the maximum value (20), then the fitness function is 0
            4. The mountain peak is [0, 0, 5.0]
            5. So the length between the creature and the mountain peak is shorter, the fitness function value is larger.
        ii. Code
            1. P1 is the coordinate value of the mountain peak
            2. P2 is the coordinate value of my creature
            3. THRESHOLD_DISTANCE is the maximum value (2)
            4. Dist0 is the distance between the P1 and the P2
            5. Dist is the fitness function value

```
    def get_distance_travelled2(self):
        if self.start_position is None or self.last_position
is None:
            return 0
        p1 = np.asarray([0, 0, 5.0])
        p2 = np.asarray(self.last_position)
        THRESHOLD_DISTANCE = 20.0
        dist0 = np.linalg.norm(p1-p2)
   dist    =    (THRESHOLD_DISTANCE-dist0)    if    dist0    <=
THRESHOLD_DISTANCE else 0
        return dist
```

    b. Experiments with changing parameters
        i. The iteration experiment
            1. The fitness function value change according to the population value

a. Fixed parameters

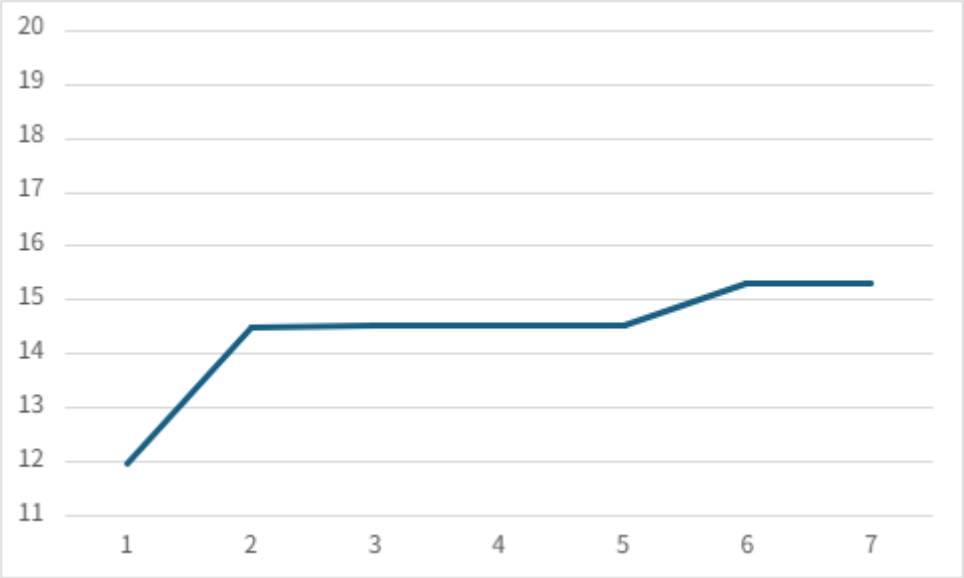
i. Gene count is 5 and the population is 20

b. Table

	Iter =0	Iter =500	Iter =1000	Iter =1500	Iter =2000	Iter =2500	Iter =3000
Fitness value	11.971	14.513	14.544	14.545	14.545	15.323	15.323

c. Graph

i. X axis is iteration. Y axis is the fitness function value



d.

e. The result

i. The fitness value is proportional to the iteration.

ii. The Population experiment

1. The fitness function value change according to the population value

i. Fixed parameters

1. Gene count is 3 and max iteration is 3000

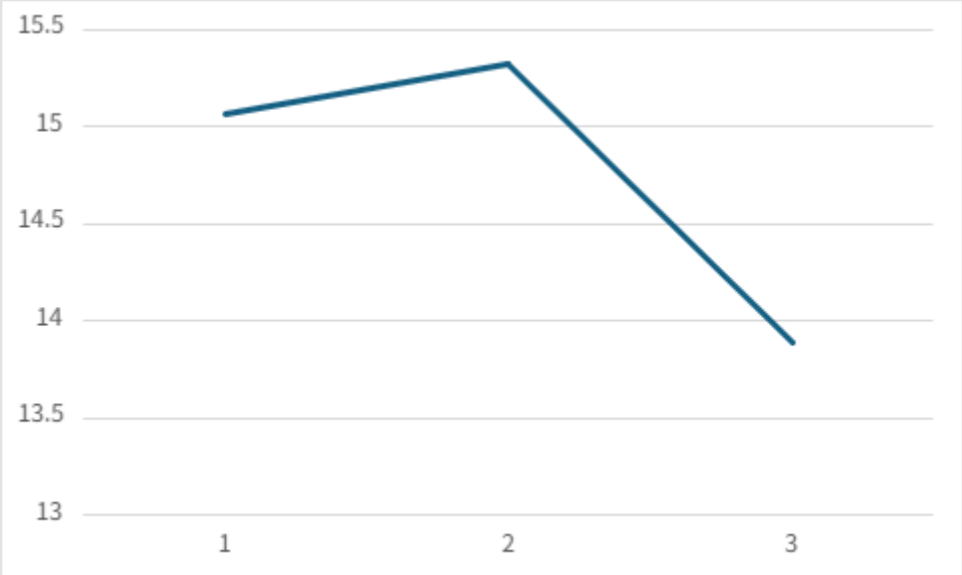
ii. table

	Population = 10	Population =20	Population =30

fitness value after max iteration	15.068	15.323	13.891

iii. Graph

1. X axis is population. And Y axis is the fitness function value



iv. The result of the experiment

1. So the best fitness value was acquired when the population was 20. So The fitness function value is not proportional to the population value.

b.

iii. Gene count experiment

1. The fitness function value change according to the gene count value

a. Fixed parameters

i. Population is 20 and max iteration is 3000

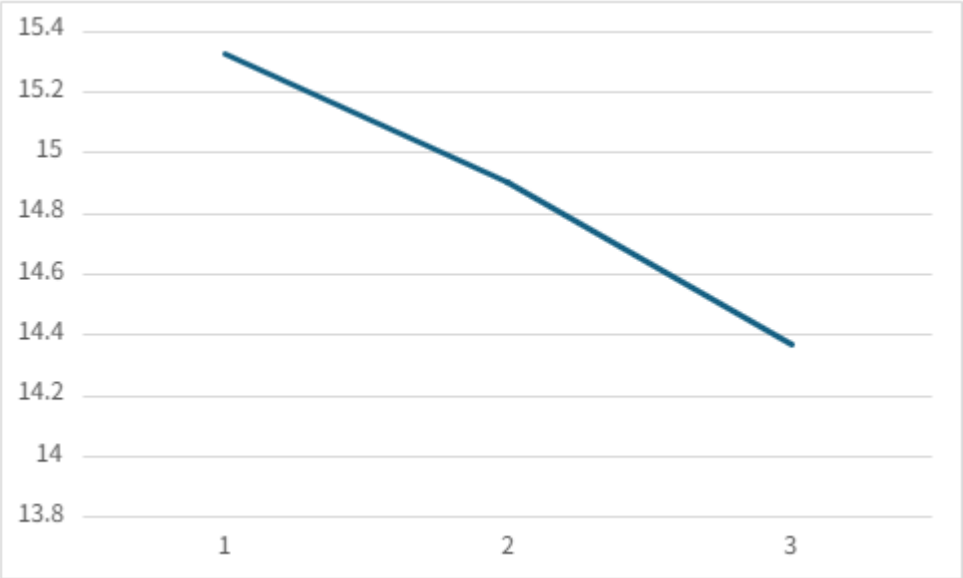
b. table

	Gene count =3	Gene count =5	Gene count =7
fitness value after max	15.323	14.9	14.369

iteration			

c. Graph

i. X axis is gene count. Y axis is the fitness value



d. The result of the experiment

i. So the best fitness value was acquired when the gene count was 3. so the fitness function is inversely proportional to the gene count value.

iv. What was the best population and the gene count combination?

1. The fitness function value change according to the combinations of population and gene count

a. Fixed parameters

i. Max iteration is 3000

b. Table

	Population =10	Population =20	Population =30
Gene count=3	15.068	**15.323**	13.891
Gene count=5	14.251	14.9	14.089
Gene count=7	14.39	14.369	14.414

c. The result of the experiment

i. The best combination is Population: 20, gene count=3

2. Experiment with encoding scheme

a. Same fitness function with basic experiments

b. Fixed body

i. There is only one body. And the Body design is fixed

ii. Code

1. So all parameters are fixed

```
        link = genome.URDFLink(name=link_name,
                parent_name=link_name,
                recur=0, #recur+1,
                link_length=1.0, #fixed gdict["link-length"],
                link_radius=0.1, #fixed gdict["link-radius"],
                link_mass=1.4, #fixed gdict["link-mass"],
                joint_type=2, #gdict["joint-type"],
                joint_parent=0, #fixed gdict["joint-parent"],
                joint_axis_xyz=2, #gdict["joint-axis-xyz"],
                joint_origin_rpy_1=2,     #gdict["joint-origin-
rpy-1"],
                joint_origin_rpy_2=2,     #gdict["joint-origin-
rpy-2"],
                joint_origin_rpy_3=2,     #gdict["joint-origin-
rpy-3"],
                joint_origin_xyz_1=2,     #gdict["joint-origin-
xyz-1"],
                joint_origin_xyz_2=2,     #gdict["joint-origin-
xyz-2"],
                joint_origin_xyz_3=2,     #gdict["joint-origin-
xyz-3"],
                control_waveform=2,            #gdict["control-
waveform"],
                control_amp=2, #gdict["control-amp"],
                control_freq=2 #gdict["control-freq"])
        )
```

c. Changeable legs

i. So the number of legs, the joint parameters of legs are changeable

ii. I designed as one gene is one leg design. So the number of gene is the number

of legs.

iii. Code

1. Link parameters are fixed (so same shape) but different joint parameters (different joint movements)

```
            link = genome.URDFLink(name=link_name,
                                   parent_name=parent_name,
                                   recur=recur+1,
                                   link_length=0.01,            #fixed
gdict["link-length"],
                                   link_radius=0.30,            #fixed
gdict["link-radius"],
                                   link_mass=0.2, #fixed gdict["link-
mass"],
                                   joint_type=gdict["joint-type"],
                                   joint_parent=0,              #fixed
gdict["joint-parent"],
                                   joint_axis_xyz=gdict["joint-axis-
xyz"],
                                   joint_origin_rpy_1=gdict["joint-
origin-rpy-1"],
                                   joint_origin_rpy_2=gdict["joint-
origin-rpy-2"],
                                   joint_origin_rpy_3=gdict["joint-
origin-rpy-3"],
                                   joint_origin_xyz_1=gdict["joint-
origin-xyz-1"],
                                   joint_origin_xyz_2=gdict["joint-
origin-xyz-2"],
                                   joint_origin_xyz_3=gdict["joint-
origin-xyz-3"],
                                   control_waveform=gdict["control-
waveform"],
                                   control_amp=gdict["control-amp"],
                                   control_freq=gdict["control-
freq"])
```

d. Experiements

i. The number of gene experiments

1. Fixed parameters

a. Max iteration is 1000. And the population is 10

2. Table

	Gene count=2	Gene count=4	Gene count=6
The fitness function value after iteration 1000	11.905	12.356	12.008

3. The result of the experiment

   a. The fitness function is not proportional to the number of genes. The best gene count was 4